

dati-basati documentation

Version 0.1.1

Contents

dati-basati	1	righetta	12
dati-basati	1	utils	13
er-diagram	2	find-side	13
entity	3	get-idx	13
subentities	4	get-next-cardinal	13
relation	5	get-prev-cardinal	13
attributes	6	get-opp-cardinal	14
_draw-attributes	6	get-aligned-coordinate	14
_draw-composite-attribute	8	update-dict	14
weak-entity	10	get-rel-position	14
_weak-entity-path	10	get-rel-position-diagonal	15
get-angles	10	get-rel-entities	15
_draw-we-path	10	add-intersect-anchor	15
custom-marks	11	draw-anchors	15
filled-circle-pk	11	get-diagonal-side	15
filled-circle-mpk	11	is-same-axis	16
filled-circle-wk	12	cardinality	16
empty-circle	12	_draw-cardinality-box	16
subentity-mark	12	entity	17
		_draw-entity-box	17

dati-basati

dati-basati

Default global settings.

Parameters

```
dati-basati(  
  fill: dictionary,  
  stroke: dictionary,  
  radius: dictionary,  
  spacing: dictionary,  
  text: dictionary,  
  attributes-position: dictionary,  
  misc: dictionary,  
  body  
) -> dictionary
```

fill dictionary

The fill of each element.

Default: (:)

stroke dictionary

The stroke of each element.

Default: (:)

radius dictionary

The radius of entities, cardinality and hierarchy.

Default: (:)

spacing dictionary

The spacing of attributes.

Default: (:)

text dictionary

Text formatting of each element.

Default: (:)

attributes-position dictionary

The position and direction of attributes.

Default: (:)

misc dictionary

Miscellaneous options.

Default: (:)

er-diagram

Draw an Entity-Relation Diagram. This is the function that wraps all the diagrams.

Parameters

```
er-diagram(  
  theme: dictionary ,  
  ..args: dictionary ,  
  body: content  
) -> content
```

theme `dictionary`

The theme of this diagram.

Default: `(:)`

..args `dictionary`

Arguments directly passed to `#cetz.canvas()`.

body `content`

Body of the diagram.

entity

Draw an entity.

Parameters

```
entity(  
  coordinates: array,  
  name: str,  
  label: str content,  
  attributes: dictionary,  
  attributes-position: dictionary,  
  primary-key: str array,  
  weak-entity: array,  
  misc: dictionary,  
  ..args: args  
) -> content
```

coordinates `array`

Coordinates of the entity.

name `str`

Unique name of the entity.

Default: `" "`

label `str` or `content`

Label for the entity (e.g. Person, Book, Employee...).

Default: `"Entity"`

attributes dictionary

Attributes of the entity (e.g. id, name, surname, address...).

Default: (:)

attributes-position dictionary

Fine tune positioning of attributes.

Default: (:)

primary-key str or array

Primary key(s).

Default: none

weak-entity array

Whether this entity is weak and the corresponding attribute. The format is as follows:

- SQL mirror: (attribute, strong entity, direction)
- With cardinal points: (cardinal pt, cardinal pt, direction)

They can be mixed.

Default: none

misc dictionary

Miscellaneous options:

- weak-entity-intersection

Default: none

..args args

Arguments passed to `#std.rect()`.

subentities

Connect subentities to the superentity.

Parameters

```
subentities(  
  hierarchy: "(t,e)" "(p,e)" "(p,o)" ,  
  entity: str,  
  subentities: array  
) -> content
```

hierarchy "(t,e)" or "(p,e)" or "(p,o)"

The hierarchy of the subentities:

- (t,e) = (total, exclusive)
- (p,e) = (partial, exclusive)
- (p,o) = (partial, overlapping)

Default: "(t,e)"

entity str

The superentity, the one to which the subentities will connect.

Default: ""

subentities array

The subentities. Can be just one.

Default: ("",)

relation

Draw a relation between (sub)entities.

Parameters

```
relation(  
    coordinates: array,  
    entities: array,  
    name: str,  
    label: str,  
    attributes: dictionary,  
    cardinality: array,  
    intersect: bool,  
    ..args: args  
) -> content
```

coordinates array

The specific coordinates of this relation. If left empty, they will be the middle point between the two entities.

Default: none

entities array

Entities that are tied by this relation.

Default: ()

name `str`

Unique name of relation.

Default: `" "`

label `str`

Label for the relation (e.g. “occupies”, “relates”, “is part of” ...).

Default: `" "`

attributes `dictionary`

Attributes of the relation (e.g. id, name, surname, address...).

Default: `( : )`

cardinality `array`

Cardinality of the given relation:

Default: `( )`

intersect `bool`

Whether to intersect the lines connecting relations instead of a direct connection.

Default: `false`

..args `args`

Arguments passed to `#std.rect()`.

attributes

_draw-attributes

Draw the attributes on a specified side of the entity.

Parameters

```
_draw-attributes(  
  start-coordinate: array,  
  entity: str,  
  position: "north" "east" "south" "west",  
  attributes: array,  
  primary-key: str,  
  dir: "ltr" "rtl",  
  drawing-mode: "from-short" "from-long",  
  in-between: dictionary,  
  padding: length,  
  label-anchor: auto "north" "east" "south" "west",  
  centered: bool,  
  attr-stroke: auto str  
) -> content
```

start-coordinate array

Starting coordinate from which the attributes will be actually drawn.

entity str

Unique name of the entity.

Default: "bella"

position "north" or "east" or "south" or "west"

Position of where the attributes will be drawn.

Default: "north"

attributes array

All attributes of the entity.

Default: none

primary-key str

A single primary key.

Default: none

dir "ltr" or "rtl"

Direction in which the attributes will be drawn. Can be either `ltr`, left to right, or `rtl`, right to left.

Default: "ltr"

drawing-mode "from-short" or "from-long"

Whether to draw starting from the smallest or longest attribute. Only applies to north, south sides: the others have always the same padding.

Default: "from-short"

in-between dictionary

Spacing in-between attributes.

Default: (x: 1.2em, y: 1.2em)

padding length

Padding from the entity. Note: this CANNOT BE different between x,y because of the weak entity path.

Default: 1.2em

label-anchor auto or "north" or "east" or "south" or "west"

Label anchor, the opposite side of there the label will be displayed. It's automatically calculated based on position and dir.

Default: auto

centered bool

Whether to align to the center the attributes: they will be drawn as if centered around the start-coordinate.

Default: false

attr-stroke auto or str

Attributes stroke.

Default: auto

draw-composite-attribute

Draw a composite attribute on a specified side.

Parameters

```
_draw-composite-attribute(  
  coordinates: array,  
  attributes: array,  
  position: "north" "east" "south" "west",  
  dimensions: int float length,  
  dimensions-coefficient: int float,  
  padding-coefficient: int float,  
  in-between: dictionary,  
  padding: length  
) -> content
```

coordinates array

From where the composite attribute will be drawn. Note that it's NOT the ellipse, rather the line LEADING TO THE CENTER of the ellipse.

attributes array

The corresponding attributes.

position "north" or "east" or "south" or "west"

Side of where the composite attribute will be drawn. Note there will not be drawn any other attributes on this same side.

Default: "west"

dimensions int or float or length

Longest radius coefficient.

Default: 0.5

dimensions-coefficient int or float

Longest radius coefficient.

Default: 5

padding-coefficient int or float

Padding of the starting line (distance from the composite attribute and the entity).

Default: 0.5

in-between dictionary

Spacing in-between attributes. Same dict as `_draw-attributes()`.

Default: (x: 1.2em, y: 1.2em)

padding `length`

Padding from the ellipse. Same dict as `_draw-attributes()`.

Default: `1.5em`

weak-entity

`_weak-entity-path`

Retrieve the path needed to draw the weak entity.

Parameters

```
_weak-entity-path(  
  start: str,  
  end: "north" "east" "south" "west",  
  dir: "CW" "CCW" "cw" "ccw"  
) -> array
```

start `str`

Starting coordinate (key, weak entity).

end `"north" or "east" or "south" or "west"`

Ending side.

dir `"CW" or "CCW" or "cw" or "ccw"`

Direction: can be clockwise or counterclockwise.

Default: `"CW"`

get-angles

Given a weak entity path, return the traversed angles.

Parameters

```
get-angles(path: array) -> array
```

path `array`

Weak entity path.

`_draw-we-path`

Draw the weak entity path.

Parameters

```
_draw-we-path(  
  name: str,  
  weak-entity: array,  
  attributes: dictionary,  
  spacing: length,  
  intersection: array  
) -> content
```

name `str`

Name of the entity.

weak-entity `array`

Weak entity in the following format: (attribute, end side, direction).

attributes `dictionary`

Entity attributes.

spacing `length`

Distance from the entity to the label(s) of the attributes (`#spacing.padding` dictionary).

intersection `array`

Specific final coordinate.

Default: `none`

custom-marks

filled-circle-pk

Filled circle for the primary keys.

Parameters

```
filled-circle-pk(fill: color) -> content
```

fill `color`

Fill colour.

Default: black

filled-circle-mpk

Filled circle for multiple primary keys.

Parameters

```
filled-circle-mpk(fill: color) -> content
```

fill color

Fill colour.

Default: black

filled-circle-wk

Filled circle for weak entities.

Parameters

```
filled-circle-wk(  
  fill: color,  
  _wk-mark-length: int float length  
) -> content
```

fill color

Fill colour.

Default: black

_wk-mark-length int or float or length

Custom length based on x,y side.

empty-circle content

Empty circle used for attributes.

subentity-mark content

Subentity mark, a custom made arrow.

righetta content

Custom made “mark”, which is just a cute line.

utils

find-side

Given a value from an array, find its key.

Parameters

```
find-side(  
    x,  
    dict  
) -> content
```

get-idx

Returns the index of x in arr.

Parameters

```
get-idx(  
    x: str,  
    arr: arrayay  
) -> int
```

x `str`

Element to search

arr `arrayay`

The array from which to search.

Default: cardinal-points

get-next-cardinal

Gets the next cardinal point (e.g. “north” -> “east”).

Parameters

```
get-next-cardinal(pt) -> str
```

get-prev-cardinal

Gets the previous cardinal point (e.g. “north” -> “west”).

Parameters

```
get-prev-cardinal(pt) -> str
```

get-opp-cardinal

Gets the opposite cardinal point (e.g. “north” -> “south”).

Parameters

```
get-opp-cardinal(pt) -> str
```

get-aligned-coordinate

Get the corrected coordinates. For instance top-left != north-west, because there is a little horizontal spacing to be added.

Parameters

```
get-aligned-coordinate(  
  entity,  
  side,  
  aligned,  
  in-between,  
  attributes-len  
) -> array
```

update-dict

Update a dictionary.

Parameters

```
update-dict(  
  old-dict,  
  new-dict,  
  are-val-dict: bool  
) -> dictionary
```

are-val-dict bool

Whether the dictionaries have values that are themselves dictionaries.

Default: false

get-rel-position

Return where is **center with respects to other**.

Parameters

```
get-rel-position(  
  center,  
  other,  
  ctx  
) -> string
```

get-rel-position-diagonal

Return where is **other with respects to center**, in CW direction.

Parameters

```
get-rel-position-diagonal(  
  center,  
  other,  
  ctx  
) -> array
```

get-rel-entities

Return where **center is with respects to multiple entities**.

Parameters

```
get-rel-entities(  
  center,  
  subentities,  
  ctx  
) -> string
```

add-intersect-anchor

Apply `#cetz.draw.hide()` to path and add an anchor intersection.0 which is the intersection between path and the line drawn from line-coord. Used for unorthodox relation positions.

Parameters

```
add-intersect-anchor(  
  path,  
  line-coord  
) -> content
```

draw-anchors

Draw the diagonal anchors.

Parameters

```
draw-anchors(  
  name,  
  padding  
) -> none
```

get-diagonal-side

Retrieve the side based on diagonal anchors.

Parameters

```
get-diagonal-side(  
    coord,  
    name,  
    ctx  
) -> str
```

is-same-axis

Return whether two elements are on the same axis.

Parameters

```
is-same-axis(  
    center,  
    other,  
    ctx  
) -> bool
```

cardinality

_draw-cardinality-box

Draw a cardinality box.

Parameters

```
_draw-cardinality-box(  
    coordinates: array,  
    name: str,  
    label: str,  
    hierarchy: bool,  
    ..args: args  
) -> content
```

coordinates array

Coordinates of the box.

name str

Its unique name to be referred as.

Default: none

label str

Its label (e.g. (0,1)).

Default: none

hierarchy `bool`

Whether this box is actually an hierarchy.

Default: `false`

..args `args`

All arguments passed to `#std.box()`.

entity

_draw-entity-box

Draw an entity box.

Parameters

```
_draw-entity-box(  
    coordinates: array ,  
    label: content str ,  
    name: str ,  
    has-weak-entity: bool ,  
    ..args: args  
) -> content
```

coordinates `array`

Coordinates.

label `content` or `str`

Label of the box.

name `str`

Unique name of this entity

has-weak-entity `bool`

Whether this entity has weak attributes.

..args `args`

Arguments passed to `#std.rect()`.